

Consultas avançadas com SQLAlchemy

Três consultas do item 5 e quatro analíticas da Etapa 1, todas montadas pela DSL

Regra de implementação

`consultas.py` e `analiticas_db.py` constroem as consultas com `select`, `join`, `where`, `group_by`, funções de janela e agregados. Esses módulos não recorrem a `text()` nem montam SQL por concatenação. As versões em `10_etapa2_verificacao.sql` e `04_analiticas.sql` servem de referência para comparar resultados.

Paciente, residente e preceptor terminam na tabela Pessoa. Quando dois desses papéis aparecem na mesma consulta, `aliased()` cria nomes separados e impede que as condições de JOIN se misturem.

Consulta	Técnica principal	Caminho
Preceptores de pacientes flamenguistas	aliases, DISTINCT e GROUP BY	/preceptores/supervisionados-flamenguistas
Último atendimento de cada paciente	row_number e eager loading	/pacientes/ultimo-atendimento
Percentual de alto risco	FILTER, CASE e outer joins	/residentes/percentual-alto-risco
Ranking de residentes	outer join e agregação	/analiticas/ranking-residentes
Preceptores por mês	WHERE para mês e HAVING para total	/analiticas/preceptores-por-mes
Plantões por unidade	grade semanal ou generate_series	/analiticas/plantoes-por-unidade
Pacientes sem alto risco	NOT EXISTS correlacionado	/analiticas/pacientes-sem-alto-risco

Preceptores ligados a pacientes flamenguistas

Atendimento já guarda paciente, residente e preceptor na mesma linha. A consulta parte dessa tabela, junta cada papel e filtra Pessoa.is_flamengo. COUNT DISTINCT evita duplicar atendimentos e residentes. O resultado informa titulação, especialidade, total de atendimentos e quantos residentes diferentes o preceptor supervisionou nesse recorte. Com o seed, quatro dos cinco preceptores aparecem.

Último atendimento de cada paciente

`row_number()` separa os atendimentos por paciente e ordena `data_hora` e `id` em ordem decrescente. A posição 1 é o registro mais recente; o `id` resolve dois atendimentos no mesmo instante. A subconsulta de ranking é reunida com a entidade Atendimento.

```
row_number() OVER (  
    PARTITION BY atendimento.id_paciente  
    ORDER BY atendimento.data_hora DESC,  
             atendimento.id_atendimento DESC  
)
```

A resposta percorre paciente, residente, preceptor, unidade e a coleção de procedimentos. `joinedload` atende os lados um e `selectinload` carrega a coleção em um segundo SELECT. Essa combinação evita N+1 sem multiplicar a consulta principal por cada procedimento. O seed devolve os cinco pacientes.

Percentual de alto risco por residente

A contagem condicional usa FILTER sobre Procedimento.nivel_risco = 'ALTO'. A base são linhas de Procedimento_Realizado; quantidade não multiplica a contagem. As junções externas mantêm residentes sem realização, e o CASE devolve zero quando o denominador é zero.

Residente no seed	Total	Alto risco	Percentual
Mariana Teles	5	2	40,00%
Karina Duarte	6	2	33,33%
Lucas Barbosa	3	1	33,33%
Olivia Prado	3	1	33,33%
Nathan Ribeiro	3	0	0,00%

Analíticas recuperadas da Etapa 1

Ranking de residentes

O outer join mantém quem ainda não realizou atendimento, com total zero. A ordenação usa o total decrescente e o nome para desempate.

Preceptores acima de um total no mês

A data recebida é truncada para o primeiro dia do mês. O filtro temporal entra no WHERE, antes de agrupar; o mínimo entra no HAVING, depois da contagem. Os parâmetros públicos são mes e minimo.

Plantões por residente em cada unidade

Escala é uma grade semanal e não possui uma data concreta. O parâmetro **projetar_no_mes** escolhe a leitura. Falso conta os slots fixos. Verdadeiro usa generate_series para criar os dias do mês atual e associa cada data ao dia_semana da escala.

Pacientes sem procedimento de alto risco

A subconsulta correlaciona Atendimento.id_paciente com o paciente externo e procura qualquer procedimento ALTO. A negação de exists retorna quem nunca teve esse evento. NOT EXISTS foi escolhido porque não herda o comportamento de NULL que pode esvaziar um NOT IN.

Conferência

Os testes verificam valores e formato das sete respostas. Há também uma inspeção estrutural das três consultas do item 5 e testes separados para as duas leituras de plantão. A medição lazy/eager fica em **GET /consultas/lazy-vs-eager**, documentada junto da camada ORM.